Стек и архитектура бота заявок ceiling-bot

Технический обзор: на чём написан бот, из каких частей состоит, где хранит данные, с какими сервисами работает, как разворачивается и проверяется. Версии — из uv.lock на 06.10.2026 (ветка main).

1. Коротко

Слой	Выбор
Язык	Python 3.12, asyncio
Telegram	aiogram 3 (long polling — без домена, TLS и входящих портов)
Хранилище	SQLite (WAL) через aiosqlite: заявки, переписка, очередь, состояние диалогов
HTTP-клиент	httpx — Trello, LLM, Groq
Конфигурация	pydantic-settings: переменные окружения, секреты как SecretStr
LLM	OpenAI-совместимый /chat/completions: DeepSeek через router.cheap, резерв — модель на Groq
Распознавание речи	GigaAM v3 (локально, ONNX int8) и Groq Whisper large-v3
Задачи и CRM	Trello REST API
Развёртывание	VPS на Linux с systemd, uv, выкладка git archive по SSH
Качество	pytest, pytest-asyncio, respx, покрытие (порог 93 %), ruff, GitHub Actions

2. Зависимости

Рабочие зависимости заданы диапазонами в pyproject.toml и закреплены точными версиями с хешами в uv.lock (установка — uv sync --frozen).

Пакет	Версия	Назначение
aiogram	3.31.0	Telegram Bot API: роутеры, фильтры, middleware, FSM, изоляция событий
aiohttp	3.14.3	Транспорт aiogram, скачивание файлов из Telegram
aiosqlite	0.22.1	Асинхронный доступ к SQLite
httpx	0.28.1	Запросы к Trello, LLM-провайдерам и Groq
pydantic / pydantic-settings	2.13.5 / 2.15.0	Настройки из окружения с проверкой значений
pytest / pytest-asyncio / pytest-cov	9.1.1 / 1.4.0 / 7.1.0	Тесты (только для разработки)
respx	0.23.1	Подмена HTTP в тестах — тесты не ходят в сеть
ruff	через uvx	Линтер: правила E, F, I; строка до 120 символов

Отдельное окружение сервиса расшифровки (на сервере в /opt/ceiling-bot-stt): gigaam на закреплённом коммите, torch для CPU, onnxruntime.

3. Устройство процесса

Бот — один процесс asyncio. Внутри работают независимые части:

Часть	Модули	Что делает
Диспетчер Telegram	app/main.py, app/bot/*	Роутеры: группа менеджеров, ответы клиента после анкеты, диалог. Middleware: приём (расшифровка голосового), защита от флуда. Состояние диалогов — в SQLite; обновления одного чата обрабатываются по очереди

Часть	Модули	Что делает
Диалог и LLM	handlers.py, dialog.py, assistant.py, prompts.py, facts.py, llm.py	Анкета и свободный текст; маршрутизатор «основная → резервная → скрипт», отключение сбоящей модели; проверка ответа модели по фактам
Очередь outbox	app/services/outbox.py	Все внешние действия — задачи в SQLite. Отдельный воркер на канал (Telegram, Trello, расшифровка), повторы 5 с … 30 мин, задачи одной заявки — по порядку
Планировщик	notifier.py, client_followup.py, report.py, inwork.py	Брошенные анкеты, уведомления, напоминания, эскалации, дайджест, сообщения клиенту, недельный отчёт — по состоянию базы, поэтому рестарт ничего не теряет и не дублирует
Сторож	app/services/health.py, systemd.py	Watchdog systemd (sd_notify), проверки моделей и очереди, алерты, /status, внешний пинг
Расшифровка	app/services/stt.py, deploy/stt/worker.py	GigaAM через Unix-сокет, Groq как резерв, отложенная и теневая расшифровка

Путь сообщения клиента

Telegram → middleware (флуд, приём, расшифровка) → обработчик анкеты или LLM → запись в базу (заявка, сообщение) и задачи в очередь в той же транзакции → воркеры отправляют в Trello и группу менеджеров.

4. Данные

- Одна база SQLite в режиме WAL, одно соединение; записи сериализуются замком, изменение и его задачи в очереди фиксируются одной транзакцией.
- Таблицы: leads (заявки и их этапы), messages (переписка), outbox (очередь задач), fsm (состояние диалогов), tg_messages (номера сообщений о заявке в группе), kv (служебные отметки).
- Схема дописывается при старте (список миграций и разовые заполнения). Бот пропускает колонки, которых не знает, поэтому откат кода обычно не требует отката базы.
- Бэкап каждую ночь в 03:30 таймером systemd, хранится 14 копий (файлы 600, каталог 700); бэкап проверяет, что копирует настоящую базу бота.

5. Внешние сервисы

Сервис	Как используется	Ограничения и защита
Telegram Bot API	Long polling, сообщения, кнопки, файлы	Темп 18 сообщений в минуту на группу; ожидание по retry_after; удаление сообщений — только 48 ч
Trello REST	Карточки, списки, метки, комментарии, вложения	Ключ и токен в параметрах запроса, логгер httpx приглушён; вложения до 10 МБ
router.cheap (DeepSeek)	Основная LLM, ответ в JSON	Таймаут 15 с; 3 ошибки подряд — отключение на 5 мин; лимиты обращений
Groq	Резервная LLM (по умолчанию openai/gpt-oss-120b) и Whisper large-v3	Тот же ключ; проверка /models раз в 5 мин без затрат токенов
GigaAM	Локальная расшифровка	Отдельный сервис без сети, запросы по одному
healthchecks.io	Внешний пинг раз в 5 минут (необязательно)	Только если задан HEALTHCHECK_URL

6. Распознавание речи

- GigaAM v3_e2e_rnnt (открытая модель Сбера), экспорт в ONNX и квантование int8 готовятся локально скриптом prepare_model.py; на сервер уходят веса с проверкой SHA256.
- Сервис ceiling-bot-stt запускается systemd-сокетом /run/ceiling-bot-stt.sock: без сети (PrivateNetwork), лимит памяти 1,4 ГБ, 1 поток, выход после 10 минут без запросов.
- Голосовое длиннее 25 секунд режется на куски по паузам; на ответ «на лету» — 25 секунд на две модели, иначе расшифровка уходит в очередь.
- Теневое сравнение (STT_SHADOW): вторая модель расшифровывает то же голосовое для статистики похожести; отчёт — app/ops/stt_compare.py.

7. Развёртывание и эксплуатация

Юнит systemd	Назначение
ceiling-bot.service	Бот: Type=notify, WatchdogSec=60, Restart=always; после аварийной остановки — алерт с причиной (ExecStopPost); DynamicUser, ProtectSystem=strict, пустой набор capabilities, фильтр системных вызовов, NoNewPrivileges
ceiling-bot-alert.service	Алерт «бот остановлен»: systemd исчерпал перезапуски (OnFailure)
ceiling-bot-backup.service / .timer	Ночной бэкап базы и алерт при сбое
ceiling-bot-stt.socket / .service	Сервис расшифровки GigaAM по требованию

- Секреты — /etc/Solaris/ceiling-bot.env (каталог 700, файл 600, владелец root); systemd читает файл до запуска сервиса от непривилегированного пользователя.
- install.sh ставит uv и Python 3.12 в /opt/ceiling-bot, системный Python не трогает.
- Выкладка: deploy/deploy.sh проверяет чистое дерево, ветку main, линтер, тесты и порог покрытия, собирает git archive из HEAD и передаёт по SSH; на сервере deploy/release.sh делает бэкап базы до замены кода, хранит прошлую версию, проверяет запуск и умеет откатываться (код или код с базой).
- Long polling: серверу нужен только исходящий доступ в интернет.

```
./deploy/deploy.sh <хост> | ./deploy/deploy.sh --check | ./deploy/deploy.sh <хост> --rollback [--with-db]
```

8. Качество и CI

- 476 автотестов; ни один не ходит в сеть: подставные сессия Telegram, доска Trello, LLM-провайдеры, HTTP через respx; .env в тестах не читается.
- Покрытие кода (строки и ветвления) — 96 %, порог 93 % задан в pyproject.toml; ниже порога выкладка не начнётся.
- GitHub Actions на каждый push и pull request: uv sync --frozen, ruff, pytest с покрытием; права только на чтение, действия закреплены по SHA. Dependabot раз в неделю.
- Регрессионные тесты безопасности — tests/test_security.py.

9. Безопасность

- Секреты регистрируются при старте и вырезаются из логов, алертов, очереди и /status.
- Телефоны маскируются перед отправкой в LLM.
- Сообщения клиента для модели — данные, не инструкции: в системный промпт его текст не попадает (ответы анкеты — строкой JSON), правило повторяется после текста клиента; в переписке для резюме угловые скобки заменены — блок <переписка> не закрыть.
- Ответ модели проверяется кодом: JSON по схеме; суммы, проценты, скидки, «бесплатно» и виды потолков — только из фактов, иначе следующая модель или скрипт. Инструментов у модели нет.
- Данные кнопок перепроверяются: владелец заявки, чат менеджеров, допустимые этапы и даты.
- Экранирование HTML в сообщениях Telegram и Markdown в карточках Trello.
- Аудит безопасности 02–03.10.2026 (скилл security-audit): 2 подтверждённые находки низкой важности и 2 профилактические правки — исправлены, покрыты тестами.

10. Настройки

Все параметры — переменные окружения; описание каждой — в .env.example.

Группа	Переменные
Telegram и чаты	BOT_TOKEN, MANAGER_CHAT_ID, OWNER_CHAT_ID, ADMIN_CHAT_ID
Рабочее время	STUDIO_TZ, WORK_START, WORK_END, WORK_DAYS, DAYS_OFF
Trello	TRELLO_API_KEY, TRELLO_TOKEN, TRELLO_BOARD_ID, TRELLO_LIST_NEW / IN_WORK / MEASURE / WON / LOST
LLM	LLM_PRIMARY_NAME / BASE_URL / API_KEY / MODEL, LLM_FALLBACK_MODEL, LLM_TIMEOUT_SEC, LLM_CALLS_PER_DAY
Расшифровка	GIGAAM_SOCKET, GIGAAM_TIMEOUT_SEC, STT_SHADOW, GROQ_API_KEY, GROQ_BASE_URL, GROQ_STT_MODEL, GROQ_STT_LANGUAGE
Сроки и напоминания	ABANDON_AFTER_MIN, REMIND_AFTER_MIN, REMIND_MAX, CLIENT_MSG_DELAY_SEC, STUCK_AFTER_MIN, MEASURE_RESULT_AFTER_MIN, THINKING_REMIND_DAYS, ESCALATE_AFTER_HOURS, ASK_CONTACT_AFTER_MIN, RATE_AFTER_MIN, WEEKLY_REPORT
Прочее	DB_PATH, LOG_LEVEL, HEALTHCHECK_URL; факты о студии — FACTS_PATH (файл facts.md)

11. Структура репозитория

Путь	Содержимое
app/main.py, config.py	Сборка приложения, запуск, настройки
app/bot/	Диалог, анкета, /order, группа менеджеров, этапы, ответы клиента, тексты, клавиатуры, промпты, факты
app/services/	Trello, очередь, уведомления и планировщик, LLM, расшифровка, сторож, отчёт, темп отправки
app/db.py, schema.py	Доступ к базе, таблицы и миграции
app/parsing.py, worktime.py	Площадь и телефон из текста; рабочее время
app/ops/	Алерт об остановке, отчёт сравнения расшифровки
deploy/	Юниты systemd, установка, выкладка и откат, бэкап; deploy/stt/ — сервис GigaAM
tests/	Автотесты; .github/ — CI и Dependabot